

Cloud-Native Auto-Scaling Web Application with CI/CD, Monitoring & Infrastructure Automation on AWS

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

Mohammad Faiza, 24B81A05P1, Sir C R Reddy College of Engineering

Ravuri venkata ramana vinay, 24B81A04F9, Sir C R Reddy College of Engineering

Ashish Madhav Gowd Pulichara, 24B81A0404, Sir C R Reddy College of Engineering

Mohammad Abdul Shaziya Farhana, 24B81A05O8, Sir C R Reddy College of Engineering

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the **Andhra Pradesh Skill Development Corporation (APSSDC)** for providing me with the opportunity to participate in this internship program. This internship has given me valuable practical knowledge and hands-on experience in cloud computing and modern DevOps practices using Amazon Web Services (AWS).

I am especially thankful to our mentor, Mrs. Sumana Bethala, for her continuous guidance, encouragement, and valuable suggestions throughout the internship. Her support and motivation helped us understand the concepts clearly and successfully complete our project.

I would also like to thank our project team members for their cooperation, coordination, and dedication. Working together as a team helped us complete different modules of the project efficiently and gain practical experience in collaborative software development.

I extend my heartfelt thanks to the faculty members of our college for their support and encouragement throughout the internship period.

Finally, I express my gratitude to everyone who directly or indirectly contributed to the successful completion of our project, "**Cloud-Native Auto-Scaling Web Application with CI/CD, Monitoring & Infrastructure Automation on AWS.**" This internship has enhanced my technical knowledge, problem-solving abilities, teamwork, and understanding of real-world cloud technologies, which will be valuable for my future career.

Thank you.

Sincerely,

Mohammad Faiza, 24B81A05P1, mohammadfaizafanar@gmail.com

Ravuri venkata ramana vinay, 24B81A04F9, vinayravuri0910@gmail.com

Ashish Madhav Gowd Pulichara, 24B81A0404, ashishpulicharla@gmail.com

Mohammad Abdul Shaziya Farhana, 24B81A05O8, shaziya290906@gmail.com

ABSTRACT

Cloud computing has become an essential technology for building scalable, reliable, and cost-effective web applications. This project, “**Cloud-Native Auto-Scaling Web Application with CI/CD, Monitoring & Infrastructure Automation on AWS,**” demonstrates how various Amazon Web Services (AWS) can be integrated to develop and manage a modern cloud-based web application.

The project focuses on automating infrastructure deployment, application deployment, monitoring, and scaling to reduce manual effort and improve system reliability. Infrastructure is provisioned using **Terraform**, enabling Infrastructure as Code (IaC) for consistent and repeatable deployments. The web application is hosted on **Amazon EC2** instances running the **Ngnix** web server. An **Application Load Balancer (ALB)** distributes incoming traffic across multiple instances, while an **Auto Scaling Group (ASG)** automatically adjusts the number of EC2 instances based on traffic demand, ensuring high availability and efficient resource utilization.

A CI/CD pipeline is implemented using **GitHub, AWS CodePipeline, AWS CodeBuild, and AWS Systems Manager (Run Command)** to automate the process of building and deploying application updates. CodeBuild prepares the deployment artifacts, and AWS Systems Manager remotely executes the deployment script on the EC2 instance without requiring direct SSH access. Monitoring and logging are carried out using **Amazon CloudWatch**, where system performance metrics, logs, dashboards, and alarms are configured to monitor the application’s health. **Amazon SNS** is used to send notifications whenever predefined thresholds are exceeded, enabling timely responses to system events.

The project also follows AWS security best practices by implementing **Virtual Private Cloud (VPC)** networking, **IAM** roles and policies, **Security Groups**, and controlled network access. These measures help ensure secure communication between cloud resources while protecting the application from unauthorized access.

Overall, this project provides practical experience in designing, deploying, automating, monitoring, and managing a cloud-native web application using AWS services. It demonstrates the benefits of automation, scalability, continuous integration and deployment, and real-time monitoring in building reliable cloud solutions that meet modern application requirements.

TABLE OF CONTENTS

1. Introduction.....	5
2. Objectives of The Project.....	6
3. Literature Survey	8
4. Proposed System & System Architecture.....	10
5. Technologies Used.....	12
6. System Implementation.....	15
7. Conclusion.....	25

1. INTRODUCTION

Cloud computing has transformed the way applications are developed, deployed, and managed. Instead of relying on physical servers and manual infrastructure management, organizations can use cloud platforms to build applications that are scalable, secure, reliable, and cost-effective. Among the available cloud service providers, **Amazon Web Services (AWS)** is one of the most widely used platforms because it offers a broad range of services for computing, networking, storage, security, monitoring, and application deployment.

Modern web applications are expected to handle varying workloads, remain highly available, provide secure access, and support frequent updates without affecting user experience. Managing these requirements manually can be time-consuming and prone to errors. To overcome these challenges, organizations adopt cloud-native technologies, Infrastructure as Code (IaC), Continuous Integration and Continuous Deployment (CI/CD), automated monitoring, and auto scaling to improve efficiency, reliability, and scalability.

The main objective of this project, "**Cloud-Native Auto-Scaling Web Application with CI/CD, Monitoring & Infrastructure Automation on AWS,**" is to design and deploy a cloud-native web application using AWS services. The project demonstrates how infrastructure can be automated, applications can be deployed continuously, system performance can be monitored in real time, and cloud resources can scale automatically based on user demand. It also aims to provide practical experience with modern cloud computing and DevOps practices.

The project uses several AWS services to build a secure and efficient cloud environment. A Virtual Private Cloud (VPC) provides network isolation, while Security Groups and IAM roles ensure secure communication and controlled access to resources. The infrastructure is created using **Terraform**, which follows the Infrastructure as Code (IaC) approach, allowing cloud resources to be provisioned automatically through configuration files instead of manual setup.

The web application is hosted on **Amazon EC2** instances running the **Nginx** web server. An **Application Load Balancer (ALB)** distributes incoming traffic across multiple EC2 instances to improve performance and high availability. An **Auto Scaling Group (ASG)** automatically increases or decreases the number of running instances based on application demand, ensuring efficient resource utilization while maintaining consistent performance.

To automate software delivery, the project implements a CI/CD pipeline using **GitHub, AWS CodePipeline, AWS CodeBuild, and AWS Systems Manager (Run Command)**. Whenever changes are pushed to the GitHub repository, AWS CodePipeline automatically starts the deployment workflow. AWS CodeBuild builds the application and prepares the deployment artifacts. After a successful build, AWS Systems Manager remotely executes the deployment script on the EC2 instance. The script copies the latest application files to the Nginx web server directory and restarts the web server, ensuring that the updated application is deployed consistently without requiring direct SSH access..

The project also includes monitoring and alerting using **Amazon CloudWatch** and **Amazon SNS**. CloudWatch collects important system metrics and logs, provides dashboards for real-time monitoring, and generates alarms when predefined thresholds are exceeded. SNS sends notification emails to administrators, enabling quick responses to system issues and helping maintain application reliability.

Security has been considered throughout the project by implementing IAM roles, Security Groups, and secure network configurations within the AWS environment. These measures help protect cloud resources while allowing only authorized communication between different components of the application.

The project was developed collaboratively by a team of four members, with responsibilities divided across infrastructure provisioning, compute and networking, CI/CD implementation, and monitoring, testing, and documentation. This collaborative approach ensured the successful completion of the project while providing hands-on experience with real-world cloud development practices.

Overall, this project demonstrates how multiple AWS services can work together to build a scalable, secure, reliable, and highly available cloud-native web application. It highlights the importance of automation, continuous deployment, monitoring, and cloud security in modern software development while providing practical knowledge of AWS and DevOps technologies that are widely used in the industry.

2. OBJECTIVES OF THE PROJECT

2.1 Main Objective

The primary objective of this project is to design, develop, and deploy a **Cloud-Native Auto-Scaling Web Application with CI/CD, Monitoring & Infrastructure Automation on AWS**. The project aims to demonstrate how various AWS cloud services can be integrated to build a scalable, secure, reliable, and highly available web application. It also focuses on automating infrastructure provisioning,

application deployment, monitoring, and resource management by following modern cloud computing and DevOps practices.

2.2 Specific Objectives

The specific objectives of the project are:

- To create a secure cloud networking environment using **Amazon Virtual Private Cloud (VPC)** with public and private subnets.
 - To configure **Security Groups** and **IAM roles** for secure communication and controlled access to AWS resources.
 - To automate infrastructure provisioning using **Terraform** by implementing the Infrastructure as Code (IaC) approach.
 - To deploy the web application on **Amazon EC2** instances running the **Nginx** web server.
 - To distribute incoming user traffic efficiently using an **Application Load Balancer (ALB)**.
 - To implement an **Auto Scaling Group (ASG)** that automatically adjusts the number of EC2 instances based on application demand.
 - To build an automated CI/CD pipeline using **GitHub, AWS CodePipeline, AWS CodeBuild, and AWS Systems Manager (Run Command)** for continuous application deployment.
- To monitor application health and infrastructure performance using **Amazon CloudWatch** dashboards, metrics, logs, and alarms.
- To configure **Amazon SNS** for sending email notifications whenever predefined alarm conditions are triggered.
 - To improve application availability, scalability, security, and deployment efficiency through cloud automation.
 - To gain practical experience in AWS cloud services, Infrastructure as Code, DevOps practices, and collaborative software development.

2.3 Scope of the Project

The scope of this project includes designing and deploying a cloud-native web application using AWS cloud services and modern DevOps practices. It covers infrastructure provisioning through Terraform, application hosting on Amazon EC2 with the Nginx web server, traffic distribution using an Application Load Balancer, automatic scaling with Auto Scaling Groups, and automated application deployment through a CI/CD pipeline. The project also includes real-time monitoring using Amazon CloudWatch and email notifications through Amazon SNS to ensure reliable system performance.

The project is developed for educational and internship purposes to provide practical experience in cloud computing, infrastructure automation, continuous deployment, monitoring, and cloud security. Although the application is implemented on a small scale using AWS Free Tier resources wherever

possible, the architecture follows industry-standard practices and can be extended for enterprise-level applications.

3. LITERATURE SURVEY

3.1 Overview of the Literature Survey

This chapter presents the theoretical background and concepts related to the proposed cloud-native web application. It explains the fundamental technologies used in the project, including cloud computing, Amazon Web Services (AWS), Infrastructure as Code (IaC), Continuous Integration and Continuous Deployment (CI/CD), auto scaling, load balancing, monitoring, and DevOps practices. Understanding these concepts provides the foundation for designing, implementing, and managing the proposed system.

3.2 Cloud Computing

Cloud computing is the delivery of computing services such as servers, storage, networking, software, and databases through the internet. Instead of investing in physical hardware, organizations can rent cloud resources whenever needed and pay only for the resources they use.

Cloud computing provides several advantages, including reduced infrastructure costs, improved scalability, high availability, automatic resource management, and faster application deployment. It also enables organizations to respond quickly to changing business requirements without making large investments in hardware.

The three main cloud service models are:

- Infrastructure as a Service (IaaS)
- Platform as a Service (PaaS)
- Software as a Service (SaaS)

AWS mainly provides Infrastructure as a Service (IaaS), allowing users to create and manage virtual machines, networking resources, storage services, and security configurations.

3.3 Amazon Web Services (AWS)

Amazon Web Services (AWS) is a cloud computing platform developed by Amazon that provides more than 200 cloud services across multiple global regions. It allows organizations to build, deploy, and manage applications using reliable and secure cloud infrastructure.

AWS offers services for computing, storage, networking, databases, monitoring, security, analytics, artificial intelligence, and application deployment. Its pay-as-you-go pricing model makes it suitable for both small-scale projects and enterprise-level applications.

In this project, AWS services such as Amazon EC2, Amazon VPC, Application Load Balancer, Auto Scaling, IAM, CloudWatch, SNS, and S3 are used to build and manage the cloud-native web application.

3.4 Infrastructure as Code (IaC)

Infrastructure as Code (IaC) is the practice of managing and provisioning cloud infrastructure using configuration files instead of manually creating resources through the cloud console.

Terraform is one of the most widely used IaC tools. It enables developers to define cloud infrastructure in code, making deployments consistent, repeatable, and easier to maintain. IaC reduces human errors, improves collaboration, and allows infrastructure to be recreated whenever necessary.

In this project, Terraform is used to provision AWS resources such as the VPC, subnets, route tables, security groups, IAM roles, Application Load Balancer, Auto Scaling Group, and EC2 instances.

3.5 Continuous Integration and Continuous Deployment (CI/CD)

Continuous Integration (CI) is the practice of automatically integrating code changes into a shared repository whenever developers commit their work. Continuous Deployment (CD) automatically deploys the updated application after successful testing and build processes.

CI/CD improves software quality by reducing manual deployment errors and enabling faster application updates.

In this project, GitHub is used to store the application source code. AWS CodePipeline manages the deployment workflow, AWS CodeBuild prepares the deployment package, and **AWS Systems Manager (Run Command)** remotely executes the deployment script on the Amazon EC2 instance. This enables automated deployment without requiring direct SSH access to the server.

3.6 Auto Scaling and Load Balancing

Auto Scaling automatically adjusts the number of running EC2 instances based on application demand. When user traffic increases, additional instances are launched automatically. During periods of low traffic, unnecessary instances are terminated to reduce costs.

An Application Load Balancer (ALB) distributes incoming user requests across multiple EC2 instances. It continuously checks the health of the instances and routes traffic only to healthy servers, improving application availability, fault tolerance, and performance.

Together, Auto Scaling and Load Balancing help maintain a highly available and scalable cloud application.

3.7 Monitoring and Logging

Monitoring is an essential part of cloud infrastructure management because it helps administrators observe system performance and detect issues before they affect users.

Amazon CloudWatch collects performance metrics such as CPU utilization, memory usage, disk activity, and network traffic. It also gathers application and system logs through CloudWatch Logs.

CloudWatch Dashboards provide a centralized view of system performance, while CloudWatch Alarms notify administrators when predefined thresholds are exceeded. Amazon SNS sends these notifications through email, allowing administrators to respond quickly to system issues.

3.8 DevOps Practices

DevOps is a software development approach that combines development and operations to improve collaboration, automation, and software delivery.

DevOps encourages continuous integration, automated testing, continuous deployment, infrastructure automation, monitoring, and feedback throughout the software development lifecycle.

The use of Terraform, GitHub, CodePipeline, CodeBuild, **AWS Systems Manager (Run Command)**, CloudWatch, and Auto Scaling in this project demonstrates the implementation of modern DevOps practices.

4. PROPOSED SYSTEM & SYSTEM ARCHITECTURE

4.1 Introduction

This chapter presents the proposed system for the cloud-native web application developed using Amazon Web Services (AWS). It describes the overall system architecture, the limitations of the existing system, and the features of the proposed solution. The chapter also explains how various AWS services and DevOps tools work together to provide automated infrastructure provisioning, continuous deployment, load balancing, auto scaling, monitoring, and security. The proposed architecture is designed to improve application scalability, availability, reliability, and operational efficiency.

4.2 Existing System

Traditional web applications are generally deployed on a single physical or virtual server. Infrastructure setup, application deployment, monitoring, and scaling are usually performed manually. This approach increases administrative effort and makes the system difficult to maintain as application traffic grows.

Limitations of the Existing System

- Manual infrastructure provisioning.
- Manual application deployment.
- Limited scalability.
- Single point of failure.
- Higher maintenance effort.
- Increased deployment errors.
- Limited monitoring and alerting.

- Poor resource utilization.

4.3 Proposed System

The proposed system overcomes the limitations of traditional deployment by adopting cloud-native architecture and DevOps practices. AWS services are integrated with automation tools to simplify infrastructure management, application deployment, monitoring, and scaling.

Terraform is used to provision the AWS infrastructure automatically using Infrastructure as Code (IaC). The web application is deployed on Amazon EC2 instances running the Nginx web server. An Application Load Balancer distributes incoming requests across multiple EC2 instances, while an Auto Scaling Group adjusts the number of instances based on user traffic. GitHub, AWS CodePipeline, AWS CodeBuild, and **AWS Systems Manager (Run Command)** automate the deployment process. CodeBuild generates the deployment artifacts, and Systems Manager remotely executes the deployment script on the EC2 instance, reducing manual effort and ensuring consistent application updates. Amazon CloudWatch monitors system performance and Amazon SNS sends alerts whenever abnormal conditions are detected.

The proposed system provides improved scalability, security, automation, high availability, and efficient resource management compared to traditional deployment methods.

4.4 System Architecture

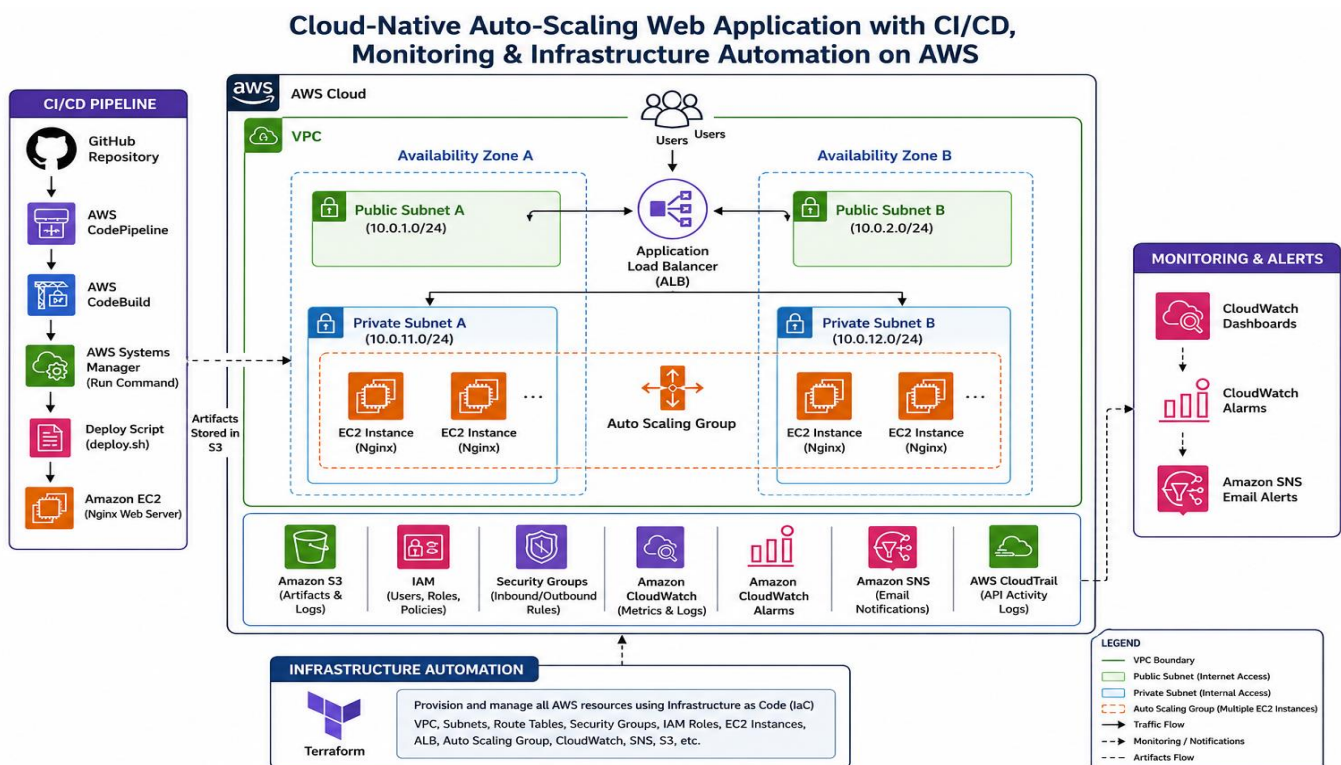


Figure 4.1: Architecture of the Proposed Cloud-Native Auto-Scaling Web Application

Figure 4.1 illustrates the overall architecture of the proposed system. Terraform provisions the AWS infrastructure, including the Virtual Private Cloud (VPC), subnets, security groups, IAM roles, Application Load Balancer, Auto Scaling Group, and EC2 instances. The application source code is stored in GitHub, and whenever changes are pushed, AWS CodePipeline automatically starts the deployment process. AWS CodeBuild prepares the deployment package, and **AWS Systems Manager (Run Command)** remotely executes the deployment script on the EC2 instance. The deployment script copies the latest application files to the Nginx web server directory, sets the required permissions, and restarts the Nginx service to make the updated application available. User requests are routed through the Application Load Balancer to healthy EC2 instances, ensuring high availability and efficient traffic distribution. Amazon CloudWatch continuously monitors system metrics and logs, while Amazon SNS sends notification emails whenever predefined alarm conditions are triggered.

4.5 Advantages of the Proposed System

The proposed system offers several advantages over traditional deployment methods:

- Automated infrastructure provisioning using Terraform.
- Secure cloud networking with Amazon VPC and Security Groups.
- Automated application deployment through CI/CD.
- High availability using the Application Load Balancer.
- Automatic scaling based on user traffic.
- Continuous monitoring using Amazon CloudWatch.
- Real-time email notifications using Amazon SNS.
- Reduced manual effort and deployment errors.
- Improved reliability and scalability.
- Easy maintenance and future expansion.

5. TECHNOLOGIES USED

The successful development of a cloud-native application requires the integration of various technologies and tools, each serving a specific purpose. The proposed system combines AWS cloud services, infrastructure automation tools, DevOps technologies, and a web server to provide a secure, scalable, and automated deployment environment. This chapter presents a brief overview of the technologies used in the project and highlights their roles in infrastructure provisioning, application deployment, networking, monitoring, security, and continuous integration and deployment.

5.1 Cloud Platform

- **Amazon Web Services (AWS):** Amazon Web Services (AWS) is the cloud computing platform used to develop and deploy the proposed cloud-native web application by providing secure, scalable, and reliable cloud services.

5.2 AWS Services

- **Amazon Virtual Private Cloud (VPC):** Amazon VPC provides an isolated virtual network that securely hosts the cloud infrastructure and enables communication between AWS resources.
- **Amazon Elastic Compute Cloud (EC2):** Amazon EC2 provides virtual servers that host the web application and run the Nginx web server.
- **Application Load Balancer (ALB):** The Application Load Balancer distributes incoming user requests across multiple EC2 instances to improve availability and performance.
- **Auto Scaling Group (ASG):** The Auto Scaling Group automatically adjusts the number of EC2 instances based on application traffic.
- **Amazon Simple Storage Service (S3):** Amazon S3 stores deployment artifacts and other project files required during the CI/CD process.

5.3 Security and Access Management

- **AWS Identity and Access Management (IAM):** AWS Identity and Access Management (IAM) controls user authentication and authorization by managing users, groups, roles, and permissions securely.
- **IAM Roles:** IAM Roles provide temporary permissions that allow AWS services to securely access other AWS resources without storing permanent credentials.
- **IAM Policies:** IAM Policies define the permissions that determine which AWS resources and actions are allowed or denied for users, groups, and roles.
- **Security Groups:** Security Groups act as virtual firewalls by controlling inbound and outbound traffic for AWS resources.

5.4 Monitoring and Logging

- **Amazon CloudWatch:** Amazon CloudWatch collects system metrics, application logs, dashboards, and alarms to continuously monitor the health and performance of AWS resources.
- **Amazon Simple Notification Service (SNS):** Amazon SNS sends email notifications whenever CloudWatch alarms detect predefined threshold violations.

- **AWS CloudTrail:** AWS CloudTrail records API activities and user actions performed within the AWS account, providing audit logs for security and compliance

5.5 Infrastructure Automation Tool

- **Terraform:** Terraform automates the provisioning and management of AWS infrastructure using the Infrastructure as Code (IaC) approach.

5.6 DevOps and CI/CD Tools

- **GitHub:** GitHub is used as the version control system to store and manage the project's source code.
- **AWS CodePipeline:** AWS CodePipeline automates the continuous integration and continuous deployment workflow.
- **AWS CodeBuild:** AWS CodeBuild compiles the source code and prepares deployment artifacts for the application.
- **AWS Systems Manager (Run Command):** AWS Systems Manager is used to remotely execute the deployment script on the EC2 instance. It automates application deployment without requiring direct SSH access. The deployment script copies the application files to the Nginx web server directory and restarts the web server after each successful build.

5.7 Programming and Scripting Technologies

- **HTML:** HTML (HyperText Markup Language) is used to create the structure and content of the web pages.
- **CSS:** CSS (Cascading Style Sheets) is used to design the layout, colors, fonts, and overall appearance of the web application.
- **JavaScript:** JavaScript adds interactivity and dynamic functionality to the web application, improving the user experience.
- **YAML:** YAML is used to define configuration files for the CI/CD pipeline, including the **buildspec.yml** file used by AWS CodeBuild and configuration files required for the deployment workflow.

5.8 Web Server

- **Nginx:** Nginx is the web server used to host and serve the web application on Amazon EC2 instances.

6. SYSTEM IMPLEMENTATION

This chapter describes the implementation of the proposed cloud-native auto-scaling web application on Amazon Web Services (AWS). It explains how the project components were integrated to create a secure, scalable, and automated cloud environment. The implementation includes infrastructure provisioning using Terraform, deployment of the web application on Amazon EC2, configuration of networking components, implementation of a CI/CD pipeline, monitoring and logging, and validation of the overall system.

6.1 Infrastructure Provisioning

6.1.1 AWS Environment Setup

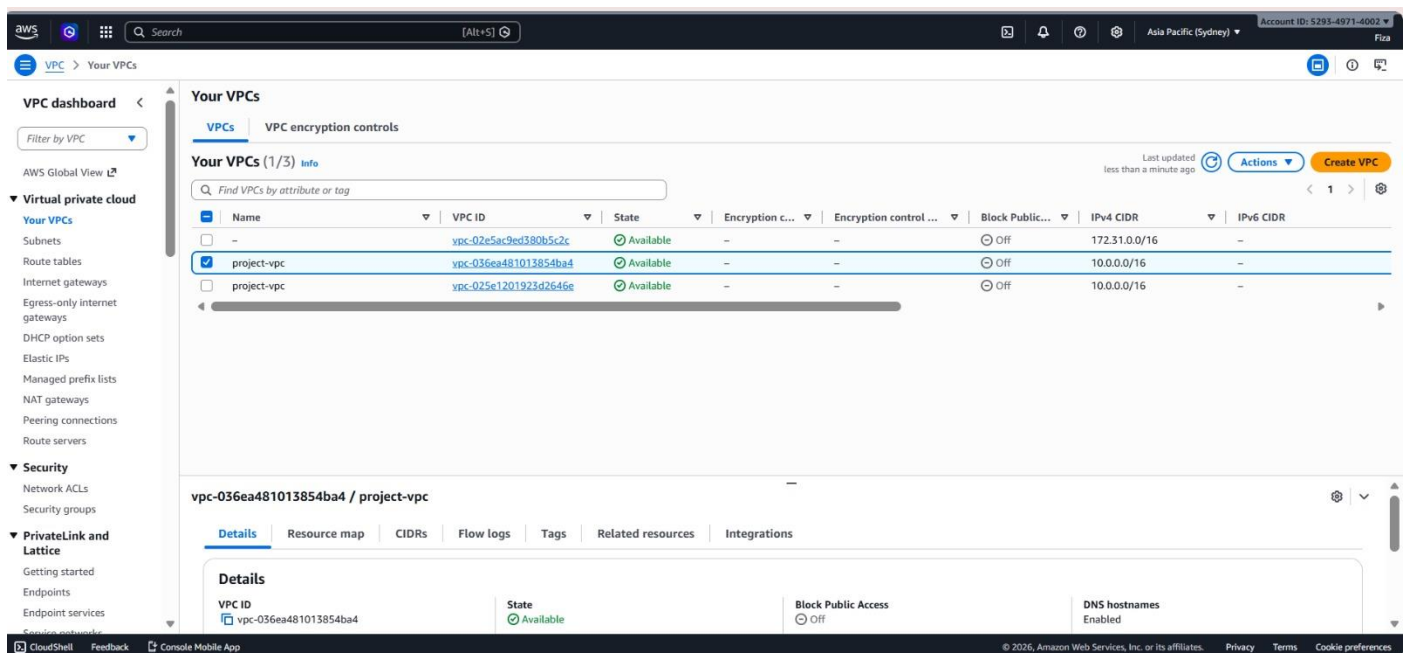
The AWS environment was prepared by configuring the required account settings, selecting the deployment region, and creating the necessary resources for the project.

6.1.2 Terraform Implementation

Terraform was used to provision and manage the AWS infrastructure automatically using Infrastructure as Code (IaC), ensuring consistent and repeatable deployments.

6.1.3 Network Infrastructure

A Virtual Private Cloud (VPC) with public and private subnets, route tables, and an Internet Gateway was implemented to provide secure and organized network communication.



Route tables (9) info

Find route tables by attribute or tag

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
☐	project-rtb-private	rtb-02d1b737b8bd665d	2 subnets	—	No	vpc-036ea481013854ba4 proj...	529349714002
☐	private-route-table	rtb-080c18e2cea5d1f7b	2 subnets	—	No	vpc-025e1201923d2646e proj...	529349714002
☐	—	rtb-01527d531c6f61bd	—	—	Yes	vpc-02e5ac9ed380b5c2c	529349714002
☐	project-rtb-db	rtb-0d8ec3194e2373646b	2 subnets	—	No	vpc-036ea481013854ba4 proj...	529349714002
☐	—	rtb-0c06fb1afda8e3a63	—	—	Yes	vpc-025e1201923d2646e proj...	529349714002
☐	public-route-table	rtb-0d850ec7009fb2dff	2 subnets	—	No	vpc-025e1201923d2646e proj...	529349714002
☐	project-rtb-private2-ap-southeast-2b	rtb-09fac8401cc10496a	—	—	No	vpc-036ea481013854ba4 proj...	529349714002
☐	—	rtb-075368e9d4cc4f7da	—	—	Yes	vpc-036ea481013854ba4 proj...	529349714002
☐	project-rtb-public	rtb-076e01895f7c9a2bb	2 subnets	—	No	vpc-036ea481013854ba4 proj...	529349714002

Select a route table

Internet gateways (1/3) info

Find internet gateways by attribute or tag

☐	Name	Internet gateway ID	State	VPC ID	Owner
☐	—	igw-01038c4a6d4ff547	Attached	vpc-02e5ac9ed380b5c2c	529349714002
☐	project-igw	igw-06b174aa925c125e6	Attached	vpc-036ea481013854ba4 project-vpc	529349714002
☒	project-igw	igw-0c7dd0424d1f95b04	Attached	vpc-025e1201923d2646e project-vpc	529349714002

igw-0c7dd0424d1f95b04 / project-igw

Details Tags

Details

NAT gateways (2) info

Find NAT gateways by attribute or tag

☐	Name	NAT gateway ID	Connectivity...	State	State message	Availability ...	Route table ID	Primary public I...	Primary private I...	Pri
☐	project-nat-gateway	nat-0e3f5d07fa4cf52bf	Public	Available	—	Zonal	—	13.55.217.197	10.0.5.154	eni
☐	project-nat-gateway	nat-0f9d71dca0c2cab16f	Public	Available	—	Zonal	—	32.236.254.220	10.0.3.81	eni

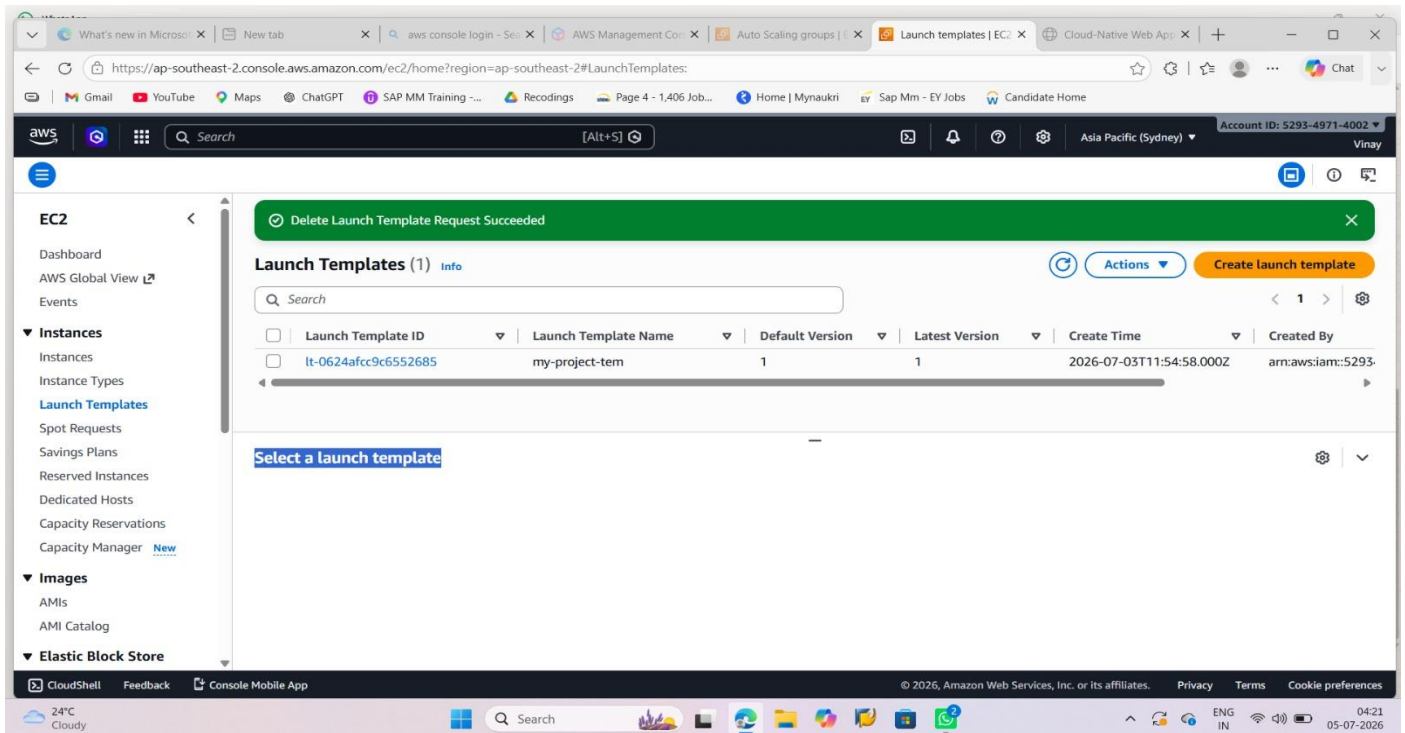
Select a NAT gateway

https://ap-southeast-2.console.aws.amazon.com/console/home?region=ap-southeast-2

6.2 Compute and Networking Implementation

6.2.1 Amazon EC2 Configuration

Amazon EC2 instances were launched to host the web application using the Ubuntu operating system.

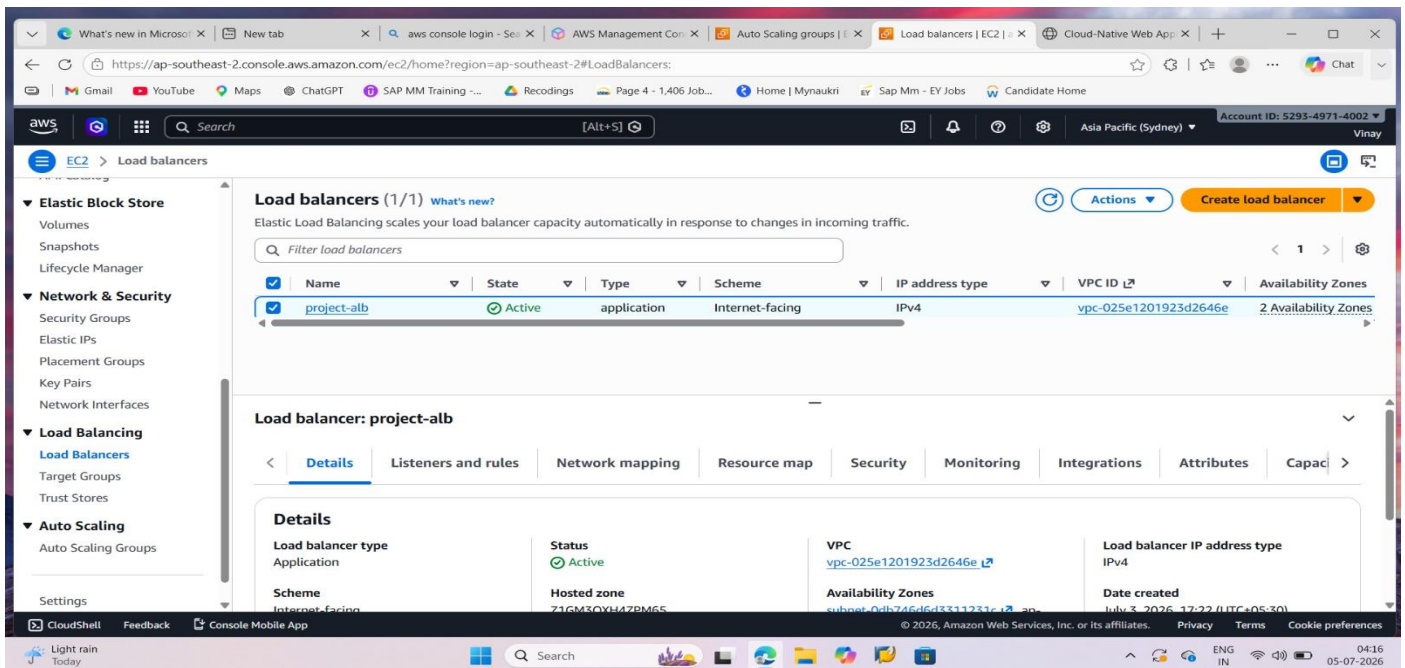


6.2.2 Nginx Web Server Configuration

The Nginx web server was installed and configured to serve the application files and handle incoming HTTP requests.

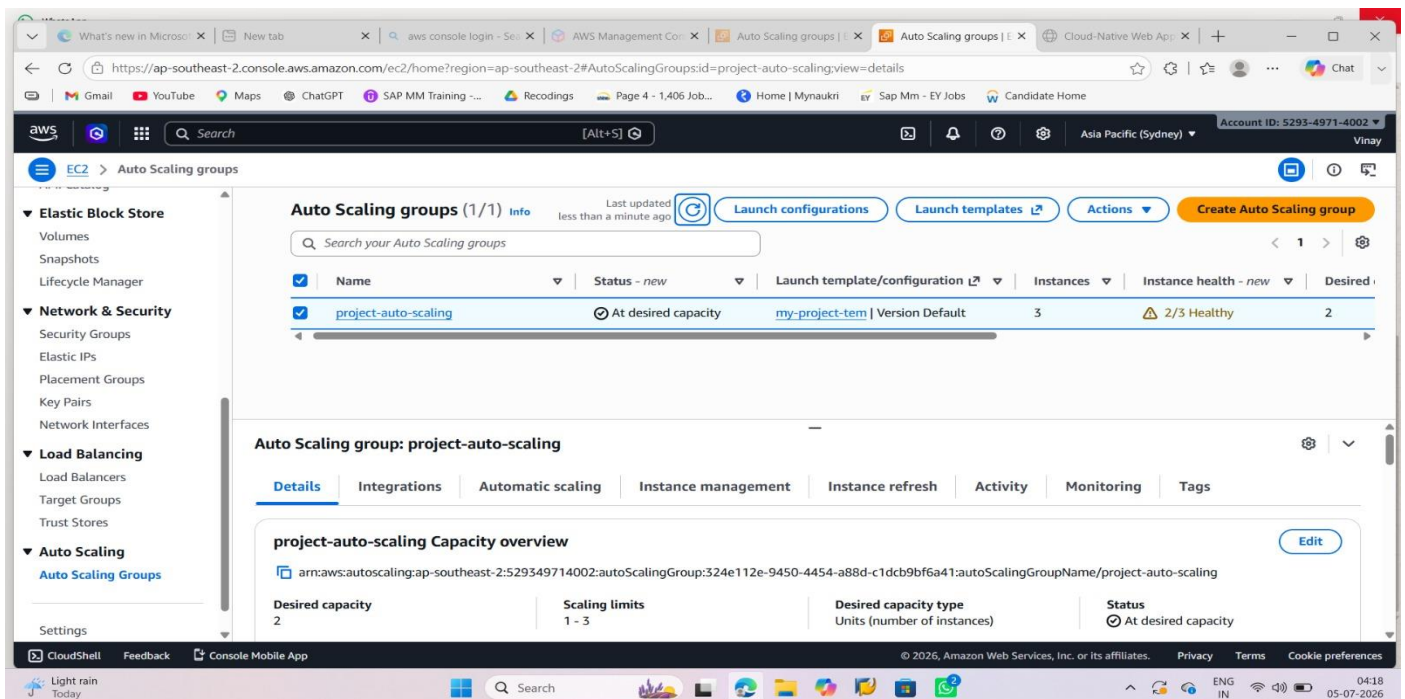
6.2.3 Application Load Balancer

An Application Load Balancer (ALB) was configured to distribute incoming traffic across the EC2 instances and improve application availability.



6.2.4 Auto Scaling Group

An Auto Scaling Group (ASG) was configured to automatically increase or decrease the number of EC2 instances based on application demand.



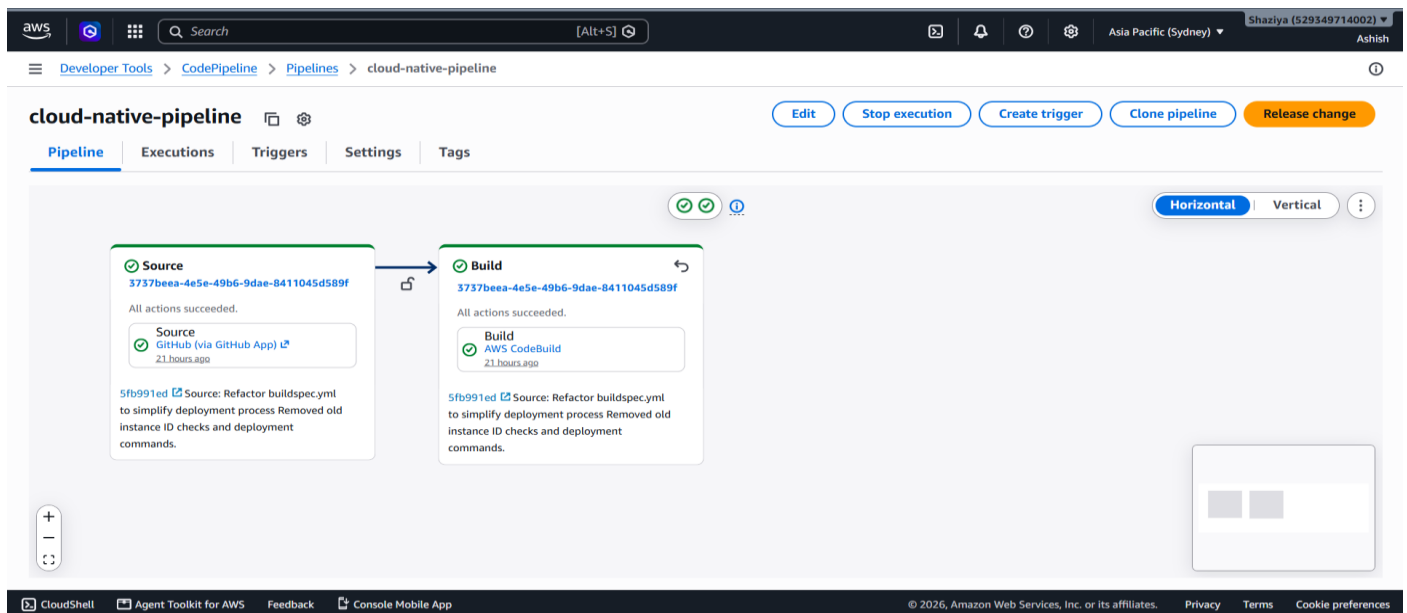
6.3 CI/CD Pipeline Implementation

6.3.1 GitHub Repository

The application source code and project configuration files were maintained in a GitHub repository for version control and collaboration.

6.3.2 AWS CodePipeline

AWS CodePipeline was configured to automate the software delivery workflow by connecting the source repository with the build process.



Name	Latest execution status	Latest source revisions	Latest execution started	Most recent executions
cloud-native-pipeline	Succeeded	Source - 5fb991ed: Refactor buildspec.yml to simplify deployment process	2 minutes ago	View details

6.3.3 AWS CodeBuild

AWS CodeBuild compiled the application and generated deployment artifacts using the buildspec.yml configuration file.

6.3.4 Amazon S3 Artifact Storage

Amazon S3 was used to store the deployment artifacts generated during the build process before deployment.

6.3.5 AWS Systems Manager (SSM)

AWS Systems Manager Run Command was used to execute deployment commands on the EC2 instance securely without requiring direct SSH access.

6.3.6 Deployment Script

A deployment script (deploy.sh) was used to copy the application files to the Nginx web directory, update file permissions, and restart the web server.

6.4 Monitoring and Logging Implementation

6.4.1 Amazon CloudWatch

Amazon CloudWatch was configured to monitor system metrics, collect logs, create dashboards, and generate alarms for resource monitoring.

The screenshot displays the AWS CloudWatch Alarms console. At the top, a green banner indicates 'Successfully created alarm High-CPU-Utilization.' with a 'View alarm' button. Below this, a blue banner states 'Some subscriptions are pending confirmation' with a 'View SNS Subscriptions' button. The main section, titled 'Alarms (1)', shows a table with one alarm: 'High-CPU-Utilization'. The alarm is in a 'Warning' state due to 'Insufficient data'. The table columns include Name, State, Actions, Actions muted - new, Last state update (UTC), and Correlation ID. The left sidebar shows the CloudWatch navigation menu with options like Ingestion, Dashboards, Alarms, AI Operations, GenAI Observability, Application Signals (APM), Infrastructure Monitoring, Logs, Metrics, Network Monitoring, and Setup. The bottom of the console shows the footer with '© 2026, Amazon Web Services, Inc. or its affiliates.' and links for Privacy, Terms, and Cookie preferences.

Name	State	Actions	Actions muted - new	Last state update (UTC)	Correlation ID
High-CPU-Utilization	Insufficient data	Actions enabled	Warning	2026-07-15 13:29:58	CPI mir

The first screenshot shows the 'Subscription: f41541b6-5a9d-4786-a8d8-5762fb776ee8' page in the Amazon SNS console. A green notification banner at the top states: 'Subscription to CloudWatch-Alerts created successfully. The ARN of the subscription is arn:aws:sns:ap-southeast-2:529349714002:CloudWatch-Alerts:f41541b6-5a9d-4786-a8d8-5762fb776ee8.' The details section shows the ARN, endpoint (shaziya290906@gmail.com), topic (CloudWatch-Alerts), and status (Pending confirmation). The subscription filter policy is currently empty.

The second screenshot shows the 'CloudWatch-Alerts' topic page. A green notification banner states: 'Topic CloudWatch-Alerts created successfully. You can create subscriptions and send messages to them from this topic.' The details section shows the topic name, ARN, display name, and type (Standard). The 'Subscriptions' tab is active, showing a table with no subscriptions found.

6.4.2 Amazon SNS

Amazon Simple Notification Service (SNS) was integrated with CloudWatch to send email notifications whenever alarm conditions were triggered.

6.4.3 AWS CloudTrail

AWS CloudTrail was enabled to record AWS API activities and maintain audit logs for security and operational monitoring.

6.5 Security and Access Implementation

6.5.1 IAM Roles and Policies

IAM roles and policies were configured to provide secure permissions for AWS services while following the principle of least privilege.

Identity and Access Management (IAM)

Search IAM

Dashboard

▼ Access Management

- Roles
- Policies
- IAM users**
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

▼ Access reports

- Access Analyzer
- Resource analysis
- Unused access
- Analyzer settings
- Credential report

Role EC2-SSM-Role created.

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

[View user](#)

IAM users (3) [Info](#)

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search

☐	User name	Path	Group	Last activity	MFA	Password age	Console last sign-in	Acc
☐	Ashish	/	0	12 minutes ago	-	50 minutes	12 minutes ago	-
☐	Fiza	/	0	-	-	10 minutes	-	-
☐	Vinay	/	0	-	-	13 minutes	-	-

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Identity and Access Management (IAM)

Search IAM

Dashboard

▼ Access Management

- Roles**
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

▼ Access reports

- Access Analyzer
- Resource analysis
- Unused access
- Analyzer settings
- Credential report

Role EC2-SSM-Role created.

[View role](#)

Roles (13) [Info](#)

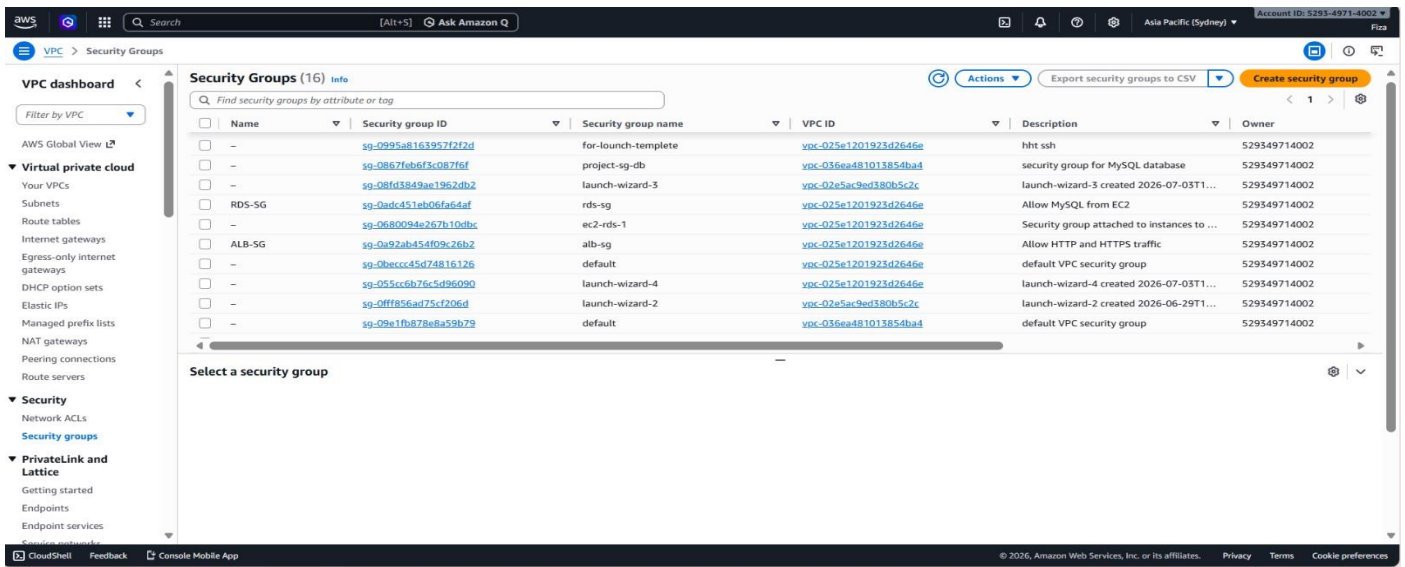
An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

☐	Role name	Trusted entities	Last activity
☐	AWSCodePipelineServiceRole-us-east-1-static-website-pipeline	AWS Service: codepipeline	7 days ago
☐	AWSServiceRoleForAutoScaling	AWS Service: autoscaling (Service-Linked Role)	1 hour ago
☐	AWSServiceRoleForElasticLoadBalancing	AWS Service: elasticloadbalancing (Service-Linked Role)	49 minutes ago
☐	AWSServiceRoleForRDS	AWS Service: rds (Service-Linked Role)	21 minutes ago
☐	AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Service-Linked Role)	43 minutes ago
☐	AWSServiceRoleForSupport	AWS Service: support (Service-Linked Role)	6 days ago
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linked Role)	-
☐	codebuild-cloud_native_codebuild-service-role	AWS Service: codebuild	6 days ago
☐	CodeBuildServiceRole	AWS Service: codebuild	-
☐	codedeploy-role	AWS Service: ec2	2 hours ago

6.5.2 Security Groups

Security Groups were implemented to control inbound and outbound network traffic and protect the cloud resources from unauthorized access.



6.6 Testing and Validation

6.6.1 Infrastructure Testing

The AWS infrastructure was verified to ensure that all resources were created successfully and functioned as expected.

6.6.2 Application Deployment Testing

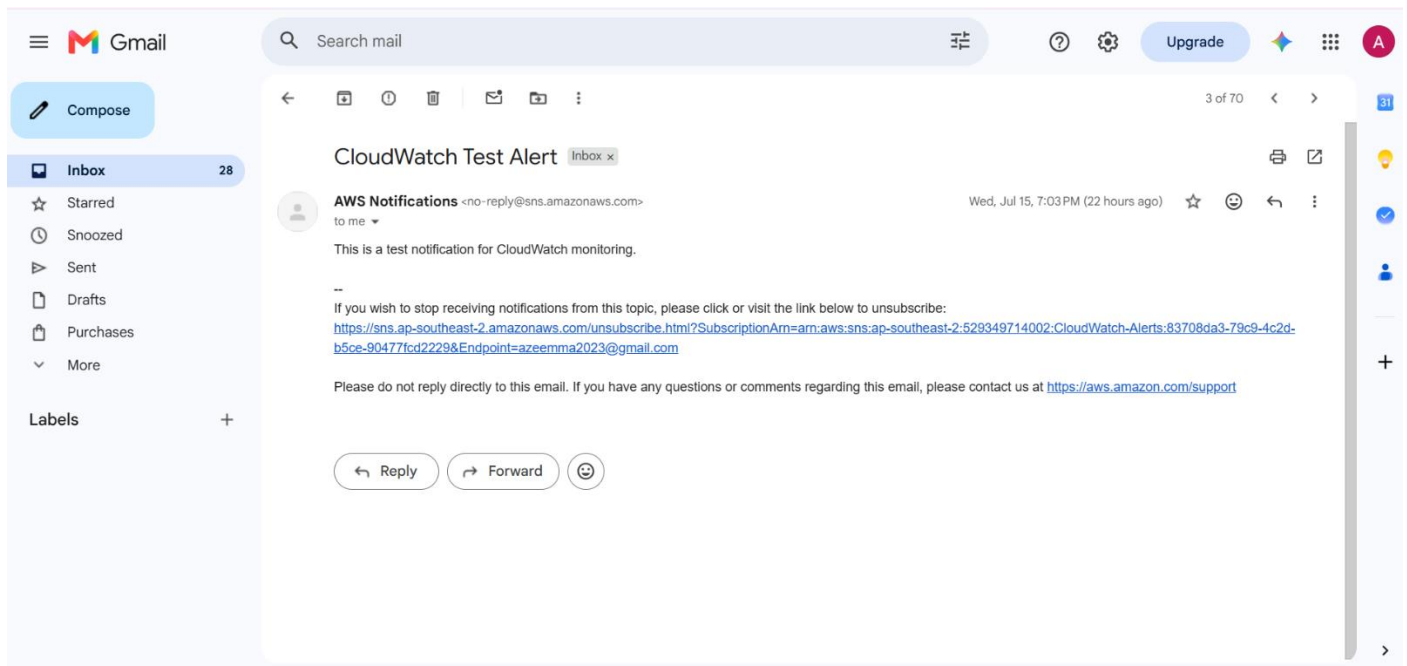
The web application deployment process was tested to verify successful deployment through the CI/CD pipeline and AWS Systems Manager.

6.6.3 Load Balancer and Auto Scaling Testing

The Application Load Balancer and Auto Scaling Group were validated to ensure proper traffic distribution and automatic scaling.

6.6.4 Monitoring and Alert Testing

CloudWatch dashboards, alarms, and SNS email notifications were tested to verify continuous monitoring and alert generation.



6.7 Challenges Faced and Solutions

During the implementation of the project, several technical challenges were encountered, including deployment limitations, IAM permission issues, configuration errors, and deployment automation. These challenges were resolved by adopting AWS Systems Manager for deployment, updating IAM roles and policies, modifying configuration files, and refining the deployment workflow.

6.8 Project Outcome

The successful implementation of the project resulted in a secure, scalable, and highly available cloud-native web application. Infrastructure provisioning was automated using Terraform, application deployment was streamlined through a CI/CD pipeline, monitoring and alerting were implemented using Amazon CloudWatch and SNS, and the overall system demonstrated reliable performance under the configured cloud environment.

This chapter presented the implementation of the proposed cloud-native web application, covering infrastructure provisioning, compute resources, networking, deployment automation, security, monitoring, and testing. The integration of AWS services and DevOps tools enabled the successful deployment of a scalable, secure, and automated cloud-native application.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The **Cloud-Native Auto-Scaling Web Application with CI/CD, Monitoring & Infrastructure Automation on AWS** was successfully designed, implemented, and deployed using a combination of AWS cloud services, Infrastructure as Code (IaC), and DevOps practices. The project demonstrated how modern cloud technologies can be integrated to build a secure, scalable, reliable, and highly available web application.

The infrastructure was provisioned using Terraform, ensuring consistent and automated deployment of cloud resources. The web application was hosted on Amazon EC2 instances running the Nginx web server, while an Application Load Balancer efficiently distributed incoming traffic across the instances. Auto Scaling Groups dynamically adjusted the number of EC2 instances based on traffic demand, improving application performance and optimizing resource utilization.

A Continuous Integration and Continuous Deployment (CI/CD) pipeline was implemented using GitHub, AWS CodePipeline, AWS CodeBuild, Amazon S3, and AWS Systems Manager (SSM).

This automated the build and deployment process, reducing manual effort and ensuring that application updates could be deployed efficiently. Monitoring and logging were achieved through Amazon CloudWatch, Amazon SNS, and AWS CloudTrail, enabling continuous observation of system performance, centralized logging, and timely notifications whenever predefined thresholds were exceeded.

The project provided valuable practical experience in cloud infrastructure provisioning, application deployment, monitoring, security, and automation. It also demonstrated the effective integration of multiple AWS services to develop a cloud-native solution that follows modern industry practices. Overall, the project successfully met its objectives and provided a strong foundation for understanding real-world cloud computing and DevOps environments.

7.2 Future Scope

Although the proposed system successfully achieves its objectives, several enhancements can be implemented in the future to further improve its functionality, scalability, and security. Some possible future improvements include:

- Containerize the application using Docker for improved portability and deployment consistency.

- Use Kubernetes or Amazon Elastic Kubernetes Service (EKS) for container orchestration and management.
- Implement a blue-green or canary deployment strategy to minimize downtime during application updates.
- Integrate automated testing into the CI/CD pipeline to improve software quality.
- Deploy the application across multiple AWS Regions to improve disaster recovery and fault tolerance.
- Enhance security by integrating AWS Web Application Firewall (WAF) and AWS Shield to protect against web attacks and distributed denial-of-service (DDoS) attacks.
- Store sensitive configuration data and credentials using AWS Secrets Manager or AWS Systems Manager Parameter Store.
- Extend monitoring capabilities by integrating advanced observability tools such as AWS X-Ray for distributed tracing.
- Support hosting of dynamic web applications with backend services and databases for more complex use cases.
- Optimize operational costs by implementing advanced scaling policies and cost-monitoring solutions.

